

# **SIGNBRIDGE PRO PROJECT DOCUMENTATION**

# TABLE OF CONTENTS

|       |                                                         |    |
|-------|---------------------------------------------------------|----|
| 1     | Project Overview .....                                  | 3  |
| 1.1   | Project Description & Purpose .....                     | 3  |
| 1.2   | Key Highlights & Value Proposition .....                | 3  |
| 2     | SRS (Software Requirements Specification).....          | 4  |
| 2.1   | Functional Requirements .....                           | 4  |
| 2.2   | Non-Functional Requirements .....                       | 4  |
| 3     | SDS (Software Design Specification) .....               | 5  |
| 3.1   | System Architecture & Tech Stack.....                   | 5  |
| 3.2   | Database Schema (sign_logs) .....                       | 6  |
| 3.2.1 | Schema Characteristics .....                            | 7  |
| 3.3   | Communication Protocols (REST & WebSockets) .....       | 7  |
| 4     | Implementation Plan.....                                | 9  |
| 4.1   | Phase 1: Environment & Tooling Setup.....               | 9  |
| 4.2   | Phase 2: Machine Learning Pipeline Development .....    | 9  |
| 4.3   | Phase 4: Frontend Development & End-to-End Wiring ..... | 10 |
| 5     | Testing Plan .....                                      | 12 |
| 5.1   | Unit Testing.....                                       | 12 |
| 5.2   | API & Integration Testing.....                          | 12 |
| 5.3   | System & Functional Testing .....                       | 12 |
| 6     | User Manual .....                                       | 13 |
| 6.1   | Prerequisites to Run Locally .....                      | 13 |
| 6.2   | Navigation & Usage Guide.....                           | 13 |

List of Tables

|                                          |   |
|------------------------------------------|---|
| Table 1: Technology Stack Summary .....  | 6 |
| Table 2: Database Schema(sign_logs)..... | 6 |
| Table 3: Core Endpoints .....            | 8 |

# 1 PROJECT OVERVIEW

---

## 1.1 *PROJECT DESCRIPTION & PURPOSE*

SignBridge Pro is a fully functional, open-source mini-demo of an AI-powered sign language interpreter built to showcase practical full-stack development and machine learning integration. At its core, the application acts as a real-time translation layer designed to bridge the communication gap between Deaf and hearing users.

The system captures sign language gestures through a standard webcam, translates them into text and speech in real time, and converts spoken replies back into live captions. By leveraging computer vision and machine learning rather than relying on scheduled human interpreters or expensive proprietary hardware, SignBridge Pro provides an accessible, around-the-clock communication tool. Furthermore, it is specifically tailored around Kenyan Sign Language (KSL), aiming to support and serve underserved communities through modern web technologies.

## 1.2 KEY HIGHLIGHTS & VALUE PROPOSITION

- **Zero Development Cost:** Built entirely using free, open-source frameworks, local development environments, and custom machine learning models, achieving a production-ready architecture with zero cloud infrastructure or hosting expenses.
- **Full-Stack & AI Skill Showcase:** Demonstrates advanced technical competence by seamlessly stitching together a modern single-page frontend, a Python computer vision/ML worker microservice, an enterprise-grade Java backend, and a relational database.
- **Instant Accessibility:** Operates entirely within standard web browsers with zero downloads, plugins, or setup queues required for the end user.

## 2 SRS (SOFTWARE REQUIREMENTS SPECIFICATION)

---

### 2.1 FUNCTIONAL REQUIREMENTS

The functional requirements outline the core behaviors, features, and operational processes that SignBridge Pro must perform to deliver its real-time translation capabilities. These have been broken down to reflect the multi-service architecture of the platform:

- **Video Capture and Streaming:** The client application must be capable of accessing the user's webcam and streaming video frames smoothly in real time without lagging the browser interface.
- **Hand Landmark Extraction:** The ML inference service must process incoming video frames using MediaPipe to accurately detect hands and extract 3D hand landmarks (21 distinct points per hand).
- **Sign Classification:** The system must utilize a trained Support Vector Machine (SVM) model to analyze the extracted landmark features, map them to known Kenyan Sign Language (KSL) classes, and output the predicted sign along with confidence probabilities.
- **Data Logging and Persistence:** The backend API must expose secure REST endpoints to receive detected sign payloads, persist each detection event into the relational database, and serve historical logs and total counts upon request.
- **Role-Based Interface Switching:** The user interface must provide a dynamic role toggle that lets users switch seamlessly between the "User client" (Deaf caller view) and the "Support agent" (customer service representative view).

### 2.2 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements define the quality attributes, system performance standards, and operational constraints of SignBridge Pro. Because this is designed as a high-performance demonstration of full-stack and AI engineering skills, these criteria ensure the system behaves reliably under real-world usage conditions:

- **Performance and Latency (NFR1):** The machine learning inference loop, landmark extraction, and WebSocket data streams must operate with minimal latency. This ensures that video feeds remain smooth and real-time captions or translations are delivered instantly without noticeable stuttering or delay.
- **Portability and Accessibility (NFR2):** The client application must run directly within modern web browsers (such as Chrome, Firefox, or Edge). The system requires no external browser plugins, native installations, or heavy software downloads for the end user.
- **Maintainability and Modularity (NFR3):** The application must follow a decoupled microservice pattern. The Angular frontend, Python ML inference worker, and Java Spring Boot backend operate independently, communicating through clear, well-defined REST and WebSocket contracts.
- **Reliability and Error Handling (NFR4):** Cross-service communication must handle dropped frames or connection interruptions gracefully. Proper CORS configurations and asynchronous threading (eventlet in Flask) must prevent worker blocks and maintain connection stability between services.

## 3 SDS (SOFTWARE DESIGN SPECIFICATION)

---

### 3.1 System Architecture & Tech Stack

SignBridge Pro is built on a modern, multi-service architecture where each component is decoupled and optimized for a specific responsibility—ranging from UI rendering and computer vision inference to enterprise data logging.

The system operates across four primary tiers:

#### 1. Frontend Client Layer

- **Technology:** Angular 17 (utilizing standalone components), TypeScript, RxJS, HTML5, and CSS3.
- **Role:** Acts as the primary user interface. It manages webcam streams, renders live video feeds (including a Picture-in-Picture layout for agent-client interactions), updates the live conversation transcript, and communicates with the backend services via WebSockets and HTTP clients.

#### 2. Machine Learning Inference Service

- **Technology:** Python 3, Flask, Flask-SocketIO (with eventlet asynchronous worker support), OpenCV (cv2), Pillow, NumPy, and scikit-learn.
- **Role:** Handles real-time computer vision processing. It captures incoming frames, utilizes the MediaPipe Hand Landmarker task to extract 3D hand coordinates (21 keypoints), scales the feature vector, and runs inference through a trained Support Vector Machine (SVM) classifier to predict Kenyan Sign Language (KSL) letters.

#### 3. Backend Logging API

- **Technology:** Java 17, Spring Boot, Maven, and Spring Data JPA.
- **Role:** Provides enterprise-grade RESTful web services. It exposes endpoints (/api/signs/detect, /api/signs/history, etc.) to process sign detection payloads, manage logging data, and return historical data to the client.

#### 4. Database Persistence Layer

- **Technology:** MySQL 8 and Hibernate ORM.
- **Role:** Persists all recognized sign events into a dedicated relational schema (sign\_logs), ensuring that session history and detection metrics can be queried or cleared on demand.

#### Technology Stack Summary Table

Table 1: Technology Stack Summary

| Tier / Component | Primary Technologies                             | Core Responsibilities                                                       |
|------------------|--------------------------------------------------|-----------------------------------------------------------------------------|
| Frontend         | Angular 17, TypeScript, RxJS, HTML5, CSS3        | User interface, webcam streaming, role toggles, Socket.IO client            |
| ML Service       | Python 3, Flask, MediaPipe, OpenCV, scikit-learn | Hand landmark extraction, SVM classification, real-time WebSocket inference |
| Backend API      | Java 17, Spring Boot, Spring Data JPA, Maven     | REST controller management, business logic, sign event logging              |
| Database         | MySQL 8, Hibernate ORM                           | Persistent storage for sign detection logs (sign_logs table)                |

3.2 DATABASE SCHEMA (SIGN\_LOGS)

The database persistence layer for SignBridge Pro relies on a relational MySQL 8 database managed via Hibernate ORM and Spring Data JPA. The primary data structure is the sign\_logs table, which captures and stores every recognized sign event detected during an active translation session.

**Table Structure:** sign\_logs

Table 2: Database Schema(sign\_logs)

| Column Name | Data Type | Constraints / Properties               | Description                                               |
|-------------|-----------|----------------------------------------|-----------------------------------------------------------|
| Id          | Long      | Primary Key, Auto-Increment (IDENTITY) | Unique identifier for each recorded sign detection event. |

| Column Name | Data Type     | Constraints / Properties    | Description                                                                     |
|-------------|---------------|-----------------------------|---------------------------------------------------------------------------------|
| signName    | String        | Non-Null (nullable = false) | The text label of the recognized sign (e.g., specific KSL alphabet letters).    |
| signEmoji   | String        | Non-Null (nullable = false) | The visual emoji or character representation associated with the detected sign. |
| Timestamp   | LocalDateTime | Non-Null (nullable = false) | The exact date and time when the sign was captured and logged by the system.    |

### 3.2.1 Schema Characteristics

- **Automatic ORM Mapping:** The entity class (SignLog.java) is mapped directly to the sign\_logs table using standard Jakarta Persistence annotations (@Entity, @Table).
- **Automated Instantiation:** When a new SignLog object is instantiated during runtime, the signName and signEmoji are assigned from the detection payload, and the timestamp is automatically initialized to the current system time (LocalDateTime.now()).
- **Repository Integration:** Data operations—such as saving new detections, retrieving entire session histories, counting total logged signs, and clearing records—are handled natively through Spring Data JPA repository extensions (SignLogRepository).

## 3.3 COMMUNICATION PROTOCOLS (REST & WEBSOCKETS)

SignBridge Pro implements a hybrid communication model, combining **WebSockets** for high-frequency, low-latency streaming and **RESTful APIs** for standard transactional operations. This split ensures that resource-heavy video frames are processed fluidly without blocking state changes or database logging.

### 1. WebSockets (Real-Time Video Streaming & Inference)

- **Protocol & Framework:** Powered by Flask-SocketIO on the Python backend and the Socket.IO client on the Angular frontend.
- **Mechanism:** To achieve smooth, real-time sign detection, the client continuously captures webcam frames and streams them over an active WebSocket connection to the ML service.

- **Event Lifecycle:**
  - connect: Establishes the socket connection and initializes available model classes.
  - frame: Transmits base64-encoded video frames from the Angular client to the Python server.
  - prediction: Emits the real-time inference results back to the client, providing the predicted sign, confidence score, and full probability mapping instantly.

## 2. RESTful APIs (Business Logic, Persistence & System Checks)

- **Protocol & Framework:** Standard HTTP/JSON architecture managed by Spring Boot REST Controllers on port 8080 and Flask REST routes on port 5000.
- **Mechanism:** Used for discrete operations that require confirmation, state changes, or database interaction.
- **Core Endpoints Summary Table:**

Table 3: Core Endpoints

| Service     | Endpoint Path         | HTTP Method | Core Functionality                                                                   |
|-------------|-----------------------|-------------|--------------------------------------------------------------------------------------|
| ML Service  | /api/predict          | POST        | Accepts a single-frame base64 image payload and returns a classification prediction. |
| ML Service  | /api/health           | GET         | Returns server health status and loaded model classes.                               |
| Backend API | /api/signs/detect     | POST        | Receives detected sign names and emojis, saving them as a new SignLog entity.        |
| Backend API | /api/signs/history    | GET         | Retrieves all stored sign log entries from the database.                             |
| Backend API | /api/signs/delete-all | DELETE      | Clears all history records from the database.                                        |
| Backend API | /api/signs/count      | GET         | Returns the total count of logged sign events.                                       |

## 4 IMPLEMENTATION PLAN

---

### 4.1 PHASE 1: ENVIRONMENT & TOOLING SETUP

Before launching the application services, the local development environment must be prepared with the necessary programming runtimes, package managers, and database tools. Because SignBridge Pro relies on a multi-service architecture, setting up each environment correctly ensures seamless local communication between the frontend, ML worker, and backend API.

- **Development Operating System:** Configured and executed within a Windows development environment.
- **Runtime & Package Manager Prerequisites:**
  - **Node.js & Angular CLI:** Required for running the frontend single-page application and managing TypeScript dependencies.
  - **Python 3:** Required for running the Flask computer vision and machine learning inference services, utilizing package managers like pip for libraries including MediaPipe, OpenCV, scikit-learn, NumPy, and Flask.
  - **Java Development Kit (JDK 17) & Maven:** Required for building and running the Spring Boot backend service.
  - **MySQL 8:** Required for housing the relational database schema (sign\_logs).
- **Initial Setup Steps:**
  1. Clone or set up the local repository containing the Angular frontend, Python scripts, and Java project files.
  2. Create and configure the local MySQL database instance on port 3306 to allow incoming connections from the Spring Boot application.
  3. Install Python dependencies using the provided requirements configuration (pip install -r requirements.txt).
  4. Build and verify the Maven dependencies for the Java backend.

### 4.2 PHASE 2: MACHINE LEARNING PIPELINE DEVELOPMENT

The machine learning pipeline is responsible for transforming raw webcam video frames into accurate Kenyan Sign Language (KSL) letter classifications in real time. Rather than processing raw pixels directly through a heavy neural network, the pipeline leverages a two-stage approach: landmark feature extraction via MediaPipe, followed by lightweight classification using a Support Vector Machine (SVM).

- **Stage 1: Hand Landmark Extraction**

- Incoming video frames are processed using the Google MediaPipe Hand Landmarker task (`hand_landmarker.task`).
- For every detected hand, MediaPipe pinpoints **21 distinct 3D hand coordinates** (incorporating X, Y, and Z axes).
- This geometric tracking condenses a complex image frame into a standardized feature vector consisting of **63 numeric coordinates** per sample.
- **Stage 2: Feature Preprocessing & Scaling**
  - The extracted 63-dimensional feature vectors are normalized using scikit-learn's `StandardScaler` to ensure that spatial variations across different hand sizes or distances from the camera do not distort predictions.
  - The dataset is structured around KSL letter classes, ensuring balanced sampling for training.
- **Stage 3: Model Training & Serialization**
  - The core classifier is built using a Support Vector Classifier (SVC) from scikit-learn, configured with a Radial Basis Function (RBF) kernel and probability estimation enabled (`probability=True`).
  - Once trained, the pipeline serializes and exports the core model assets into the local `/model` directory using Python's `pickle` library:
    - `svm_model.pickle`: The trained classifier weights.
    - `scaler.pickle`: The fitted feature scaler parameters.
    - `classes.pickle`: The array of target sign class labels.
- **Stage 4: Real-Time Inference Integration**
  - The Flask server (`app.py` / `app_socketio.py`) loads these `.pickle` assets on startup.
  - During an active session, frames streamed via WebSockets or HTTP POST requests are passed through the landmarker, scaled, and evaluated by the SVM model to return the predicted sign and top confidence probabilities instantly.

### 4.3 PHASE 4: FRONTEND DEVELOPMENT & END-TO-END WIRING

The final implementation phase focuses on building the Angular single-page application (SPA) and wiring it up with the Python ML inference service and the Java Spring Boot backend to create a cohesive, end-to-end user experience.

- **UI & Component Design** (`app.html`, `app.css`):
  - Developed a dual-view interface consisting of an engaging, responsive landing page and an immersive workspace translation hub.

- The landing page incorporates product overviews, impact statistics, how-it-works guides, offerings, and testimonials styled with custom CSS variables, keyframe animations, and a modern color palette.
- The workspace features a live video frame layout supporting a main feed (#mainVideo) and a Picture-in-Picture (PIP) feed (#pipVideo), a progress tracker pipeline, and a real-time conversation transcript panel.
- **Role Management & State Handling (app.html):**
  - Implemented an interactive role toggle allowing users to switch dynamically between the **"User client"** (Deaf caller view) and the **"Support agent"** (customer service representative view).
  - The UI state adapts to the selected role, presenting sign-recording controls for the client and audio-reply or transcription statuses for the agent.
- **End-to-End Wiring & Communication:**
  - Configured Angular services to manage WebSocket connections via Socket.IO, enabling high-frequency streaming of webcam video frames to the Flask ML worker on port 5000.
  - Wired HTTP clients to send detection results and fetch session history from the Spring Boot backend API on port 8080, ensuring actions like logging a sign or clearing history interact directly with the MySQL database.
  - Integrated developer tooling panels within the workspace to let developers test the API directly, inspect model outputs, and reset tracking metrics during demonstrations.

## 5 TESTING PLAN

---

### 5.1 UNIT TESTING

Unit testing ensures that individual components, services, and repository layers of the backend application operate correctly in isolation.

- **Backend Validation:** The Spring Boot application includes automated unit testing configured via JUnit (`SignTranslatorApplicationTests.java`) to verify that the application context loads successfully without configuration or dependency errors.
- **Repository Testing:** Data access logic is validated by testing custom JPA repository methods—such as `findBySignName(String signName)`—ensuring that database queries correctly filter and return expected sign log entities.

### 5.2 API & INTEGRATION TESTING

API and integration testing verifies that the communication channels between the disparate services (Angular frontend, Python Flask ML worker, and Java Spring Boot backend) function cohesively.

- **ML Endpoint Verification:** Python-based test scripts (`test_simple.py`, `test_angular_image.py`) are utilized to simulate live image payloads, sending base64-encoded frames to the Flask inference server (`http://localhost:5000/api/predict` and `/api/health`) to ensure valid JSON responses and proper error handling.
- **WebSocket Communication Testing:** The socket connection is tested to ensure that real-time event emitters (connect, frame, and prediction) transmit video streams and classification payloads smoothly between the client and the Flask-SocketIO worker.
- **Backend REST Testing:** Endpoints exposed by the Spring Boot server (`/api/signs/detect`, `/api/signs/history`, and `/api/signs/count`) are tested to confirm that data is accurately processed and persisted to the MySQL database.

### 5.3 SYSTEM & FUNCTIONAL TESTING

System and functional testing validates the end-to-end user experience, hardware integration, and overall application behavior within real browser environments.

- **Webcam & Media Access Testing:** Evaluates browser permission prompts, stream initialization, and video rendering across major web browsers to ensure the video pipeline functions smoothly.
- **Role-Switching and UI Responsiveness:** Functional tests are performed on the Angular frontend to verify that toggling between the "User client" and "Support agent" views correctly updates the interface elements, recording buttons, and transcript states.
- **Real-Time Tracking & Debugging:** Uses local debugging scripts (`test_webcam.py`, `test_webcam_debug.py`) to test hand-tracking stability, confidence thresholds, and model output accuracy under live camera conditions.

## 6 USER MANUAL

---

### 6.1 PREREQUISITES TO RUN LOCALLY

Because SignBridge Pro relies on a distributed multi-service architecture, your local machine must have the appropriate software runtimes, package managers, and databases installed before starting the application. Ensure the following components are set up:

- **MySQL 8 Database:** A local MySQL instance running on port 3306 with a configured schema (sign\_logs) to handle data persistence for the backend API.
- **Java Development Kit (JDK 17) & Maven:** Required to compile and run the Spring Boot backend logging service.
- **Python 3 & Pip:** Required to run the Flask machine learning inference server. Key Python packages include mediapipe, opencv-python, numpy, scikit-learn, flask, flask-cors, and pillow.
- **Node.js & Angular CLI:** Required to install dependencies and run the Angular single-page frontend application.

### 6.2 NAVIGATION & USAGE GUIDE

Once all prerequisites are installed and services are running, follow these steps to navigate and operate SignBridge Pro:

#### 1. Launching the Services

1. **Database:** Ensure your MySQL database service is active and accessible.
2. **Backend API:** Start the Spring Boot backend service on port 8080 (it will connect automatically to the MySQL database).
3. **ML Inference Service:** Navigate to the ML backend directory and launch the Flask server on port 5000 (e.g., executing `python app.py`).
4. **Frontend App:** Navigate to the Angular project directory, run `ng serve`, and open your browser at `http://localhost:4200`.

#### 2. Landing Page Navigation

- Upon visiting `http://localhost:4200`, you will be greeted by the SignBridge Pro landing page.
- Scroll through the overview sections to explore the project manifesto, impact metrics, "How It Works" guide, and testimonials.
- Click the "**Launch hub**" or "**Start Translating**" button in the navigation bar or hero section to enter the translation workspace.

#### 3. Using the Translation Hub (Workspace)

- **Role Selection:** Once inside the hub, use the role toggle at the top right to switch between "**User client**" (Deaf caller view) and "**Support agent**" (customer service representative view).
- **Connecting the Session:** Click the "**Connect call**" button to initialize the live session tools.
- **Sign Translation (Client Mode):** Click "**Start signing**" to grant browser webcam access. Position yourself in front of the camera and perform sign gestures; the system will track your hand landmarks, process the frame, and output the recognized sign and live transcript updates instantly.
- **Reviewing Session History:** Scroll down to the "**Session history**" panel to view a chronological log of all recognized signs alongside their timestamps. If needed, click the "**Clear all**" button to wipe the history clean.
- **Developer Tools:** Expand the "> **Developer tools**" accordion at the bottom of the page if you wish to run direct API checks, test the ML backend, or reset tracking memory.